High Level Design (HLD) / System Design Roadmap — 2 Hours/Day

Goal

LLD roadmap khatam hone ke baad (Sep 30, 2026), core distributed-systems/system-design concepts seekhna aur classic + real interview problems hands-on design karna — Easy → Medium → Hard, khatam ek Capstone system par jo full interview-format write-up ke sath ho.

Key Decisions

- Starts **Thursday, Oct 01, 2026** (LLD ke agle available working day se)
- **2 hours/day, Sunday off** — koi day Sunday ko schedule nahi hota
- **One system per day rule:** kisi bhi din 1 se zyada alag system design nahi hoga. Agar ek din mein multiple items hain (WE/PT/CH/Deliverable), wo sab usi din ke ek hi target system ke parts/sub-tasks honge, alag-alag system nahi.
- **Weekly structure (har milestone/week mein):** har week ke andar **3 Worked Examples + 3 Practice Tasks + 3 Challenges + 1 Deliverable** hona chahiye (total 10 items, 6 working days ke andar spread).
- **Worked Example (WE):** resource ka already-solved example study karo aur apne words + diagram mein reproduce karo (guided).
- **Practice Task (PT):** us week ke "target system" par chhota design decision — semi-guided, resource ko reference ki tarah use karte hue.
- **Challenge (CH):** usi target system ka harder/edge-case part — pehle khud (bina reference dekhe) solve karne ki koshish, phir compare karo.
- **Deliverable:** week ke saare PT + CH ka target system ke liye polished, combined design write-up.
- ■■ **Hackathon gap:** confirmed hackathon **Oct 19–20, 2026** — 2 din is roadmap se skip (auto-adjusted, manual shift nahi karna).
- Total: **48 working days (~9.5 calendar weeks)**, ending **Friday, Nov 27, 2026**
- Language-agnostic — diagrams + tradeoff reasoning primary hai, code sirf jaha zaroori ho (pseudo-code/API contracts)

Resource Map (sab 15 repos, curated by Tauseef Fayyaz — har ek ka use case)

#	Resource	What it's good for	Where used
1	donnemartin/system-design-primer	**Primary backbone** — core concepts + worked interview solutions	Har week ke WE days, Weeks 7-8
2	ashishps1/awesome-system-design-resources	Clean topic index, Top 15 Tradeoffs, Easy/Medium/Hard problem list	Har week WE/PT/CH, Week 6, Weeks 7-8
3	InterviewReady/system-design-resources (Gaurav Sen)	Deep-dives — caching internals, Paxos/Raft, authorization	Week 4, Week 5
4	binhnguyennus/awesome-scalability	Real company case studies (Netflix, Uber, LinkedIn)	Case-study reading, Weeks 1-6
5	alex/what-happens-when	Networking fundamentals — URL hit karne par kya hota hai	Day 1 (WE1)
6	arpit20adlakha/Data-Structure-Algorithms-LLD-HLD	Consolidated coding+LLD+HLD index	Backup reference, as-needed
7	kilimchoi/engineering-blogs	Curated company engineering blogs	Alt case-study source, Weeks 1-6
8	codersguild/System-Design	Chapter-wise HLD notes	Backup reading if primer/ashishps1 topic thin lage

#	Resource	What it's good for	Where used
9	arpit20adlakha/Computer-Science-Papers-For-System-Design	Foundational CS papers (consensus, distributed systems theory)	Week 5 Challenge (Paxos/Raft background)
10	addyosmani/essential-js-design-patterns	LLD design patterns reference	Week 6 (architecture ↔ design pattern connection)
11	CodingChallengesFYI/SharedSolutions	Real, buildable coding challenges	**Challenge resource** — Weeks 2, 4 (rate limiter, LRU cache builds)
12	Chalarangelo/30-seconds-of-code	Quick code snippets	Pseudo-code/API contract writing, Week 2, Week 7
13	design-gurus/grokking-system-design	Free companion repo (pattern cheat-sheets, interview framework, 40+ problem walkthroughs) to the paid Grokking course — verified, corrected from image's "design-guru"	Optional extra problem picks, Weeks 7-8
14	yangshun/tech-interview-handbook	General interview prep, behavioral	Day 48 — capstone presentation polish
15	kdn251/interviews	DS/Algo interview prep	Parallel-track resource (outside the 2hr HLD block) if coding-round prep bhi chal raha ho

Daily Time Split (120 min)

Block	Time
Read core concept / study worked example	25-30 min
Case study / real-world reading (1 article)	15-20 min
Design exercise (PT/CH/Deliverable — diagram + written tradeoffs)	55-65 min
Notes + revision checklist	10 min
Git commit	5 min

WEEK 1 — Foundations + Core Concepts (Days 1-6)

Milestone Deadline: Thursday, Oct 01, 2026 → Wednesday, Oct 07, 2026

Target system for PT/CH/Deliverable: Distributed Session Store

Day 1 — [WE1] Networking Foundations

Deadline: *Thursday, Oct 01, 2026*

Study: what-happens-when — DNS resolution → TCP handshake → HTTP request/response

- Worked Example: reproduce the repo's own "what happens when you hit enter" trace, in your own words + a diagram

`Commit: hld-day-01-networking-foundations`

Day 2 — [WE2] Scalability, Availability, Reliability, SPOF

Deadline: *Friday, Oct 02, 2026*

Study: system-design-primer — Scalability section; ashishps1 — Scalability/Availability/Reliability/SPOF

- Worked Example: reproduce Reliability vs Availability distinction with a concrete example

`Commit: hld-day-02-scalability-availability`

Day 3 — [WE3] CAP Theorem + PACELC

Deadline: *Saturday, Oct 03, 2026*

Study: system-design-primer — CAP theorem; ashishps1 — CAP Theorem page

- Case study: 1 article from awesome-scalability's Consistency section
- Worked Example: reproduce CAP explanation with your own concrete example system (not the textbook one)

`Commit: hld-day-03-cap-pacelc`

Day 4 — [PT1 + PT2] Session Store: Consistency + Availability Decisions

Deadline: *Monday, Oct 05, 2026*

Read: system-design-primer — Consistency patterns, Availability patterns

- PT1: pick strong vs eventual consistency for the Distributed Session Store, justify
- PT2: design a failover pattern (replication) for the session store nodes

`Commit: hld-day-04-session-store-consistency-availability`

Day 5 — [PT3 + CH1] Session Store: Consistent Hashing + Partition Challenge

Deadline: *Tuesday, Oct 06, 2026*

Read: ashishps1 — Consistent Hashing page

- PT3: draw a consistent hashing ring for a 4-node session store cluster, show what happens when a node is added
- CH1 (harder, khud pehle solve karo): a network partition hits the session store — reason through it with PACELC before checking any reference

`Commit: hld-day-05-session-store-hashing-partition`

Day 6 — [CH2 + CH3 + Deliverable] Session Store: Fault Tolerance + Final Doc

Deadline: *Wednesday, Oct 07, 2026*

Read: ashishps1 — Fault Tolerance, Failover pages

- CH2 (unguided): simulate a node failure — what breaks, how does rebalancing work?
- CH3 (unguided): propose a multi-region disaster-recovery plan for the session store

- **Deliverable:** compile an Availability & Consistency Decision Doc for the Distributed Session Store (CAP choice + consistency pattern + hashing scheme + failover/DR plan)

```
Commit: hld-day-06-session-store-deliverable
```

WEEK 2 — Networking & API Layer (Days 7-12)

Milestone Deadline: Thursday, Oct 08, 2026 → Wednesday, Oct 14, 2026

Target system for PT/CH/Deliverable: Public API for a Ride-Hailing App

Day 7 — [WE1] DNS + Load Balancing

Deadline: *Thursday, Oct 08, 2026*

Study: system-design-primer — DNS, Load balancer sections

- Worked Example: reproduce L4 vs L7 load balancing distinction — when to use which

`Commit: hld-day-07-dns-load-balancing`

Day 8 — [WE2] Reverse Proxy + API Gateway

Deadline: *Friday, Oct 09, 2026*

Study: system-design-primer — Reverse proxy; ashishps1 — API Gateway

- Worked Example: reproduce request flow diagram — Gateway → LB → Reverse Proxy → App servers

`Commit: hld-day-08-reverse-proxy-gateway`

Day 9 — [WE3] CDN

Deadline: *Saturday, Oct 10, 2026*

Study: system-design-primer — CDN section; ashishps1 — CDN page

- Case study: 1 CDN-related article from awesome-scalability
- Worked Example: reproduce push vs pull CDN pattern with a concrete example

`Commit: hld-day-09-cdn`

Day 10 — [PT1 + PT2] Ride-Hailing API: Contract + Style Decision

Deadline: *Monday, Oct 12, 2026*

Read: ashishps1 — REST vs GraphQL, REST vs RPC pages

- PT1: design REST endpoints for ride booking (use 30-seconds-of-code as a quick reference when writing example request/response snippets)
- PT2: decide REST vs GraphQL vs RPC for this API, justify

`Commit: hld-day-10-ride-api-contract`

Day 11 — [PT3 + CH1] Ride-Hailing API: Real-Time Tracking

Deadline: *Tuesday, Oct 13, 2026*

Read: ashishps1 — WebSockets, Webhooks pages; system-design-primer — communication section

- PT3: design a WebSocket channel + message contract for real-time trip tracking
- CH1 (unguided): work through reconnect/backpressure edge cases at scale before checking references

`Commit: hld-day-11-ride-api-realtime`

Day 12 — [CH2 + CH3 + Deliverable] Ride-Hailing API: Idempotency + Rate Limiter

Deadline: *Wednesday, Oct 14, 2026*

Read: ashishps1 — Idempotency, Rate Limiting pages

- CH2 (unguided): design idempotency keys for the booking endpoint before checking the reference
- CH3 (practical build): implement a token-bucket rate limiter as a real coding challenge (use CodingChallengesFYI/SharedSolutions for a rate-limiter challenge reference)

- **Deliverable:** full Rate Limiter + API Gateway design doc for the Ride-Hailing API (algorithm choice + justification + diagram)

```
Commit: hld-day-12-ride-api-deliverable
```

WEEK 3 — Database Layer (Days 13-18)

Milestone Deadline: Thursday, Oct 15, 2026 → Friday, Oct 23, 2026 *(Oct 19-20 = hackathon, skipped)*

Target system for PT/CH/Deliverable: Ride-Tracking App Database

Day 13 — [WE1] ACID + SQL vs NoSQL

Deadline: *Thursday, Oct 15, 2026*

Study: system-design-primer — RDBMS/ACID; ashishps1 — SQL vs NoSQL

- Worked Example: pick 2 systems from your LLD capstones, decide SQL or NoSQL for each, justify

`Commit: hld-day-13-acid-sql-nosql`

Day 14 — [WE2] Database Indexes + Bloom Filters

Deadline: *Friday, Oct 16, 2026*

Study: ashishps1 — Database Indexes, Bloom Filters pages

- Worked Example: reproduce how a B-tree index speeds up a query, in your own words + diagram

`Commit: hld-day-14-indexes-bloom-filters`

Day 15 — [WE3] Replication

Deadline: *Saturday, Oct 17, 2026*

Study: system-design-primer — master-slave/master-master replication

- Worked Example: reproduce a replication topology diagram for a read-heavy system

`Commit: hld-day-15-replication`

Hackathon: Oct 19-20, 2026 — no roadmap work these 2 days

Day 16 — [PT1 + PT2] Ride-Tracking DB: Sharding + Federation

Deadline: *Wednesday, Oct 21, 2026*

Read: system-design-primer — Sharding, Federation sections

- PT1: pick a sharding key for the trip table, justify
- PT2: decide if/where federation makes sense for related tables

`Commit: hld-day-16-ride-db-sharding-federation`

Day 17 — [PT3 + CH1] Ride-Tracking DB: Denormalization + Tuning Challenge

Deadline: *Thursday, Oct 22, 2026*

Read: system-design-primer — Denormalization, SQL tuning; ashishps1 — Database Scaling

- Case study: 1 article from awesome-scalability's Database section
- PT3: decide denormalization for read-heavy trip-history queries
- CH1 (unguided): given a slow query on this system, redesign indexes before checking references

`Commit: hld-day-17-ride-db-denormalization-tuning`

Day 18 — [CH2 + CH3 + Deliverable] Ride-Tracking DB: Architecture + Scaling Plan

Deadline: *Friday, Oct 23, 2026*

Read: ashishps1 — Database Architectures, Database Types pages

- CH2 (unguided): pick a DB architecture type (relational / wide-column / time-series) for high-frequency location pings
- CH3 (unguided): design the full replication + sharding scaling plan under 10x load
- **Deliverable:** full schema + scaling plan write-up for the Ride-Tracking App Database

```
Commit: hld-day-18-ride-db-deliverable
```

WEEK 4 — Caching Layer (Days 19-24)

Milestone Deadline: Saturday, Oct 24, 2026 → Friday, Oct 30, 2026

Target system for PT/CH/Deliverable: Product Catalog Caching Layer

Day 19 — [WE1] Caching 101 + Strategies

Deadline: *Saturday, Oct 24, 2026*

Study: system-design-primer — Client/CDN/Web server/Database/Application caching; ashishps1 — Caching Strategies

- Worked Example: reproduce cache-aside vs write-through vs write-behind tradeoffs table

`Commit: hld-day-19-caching-strategies`

Day 20 — [WE2] Cache Eviction Policies

Deadline: *Monday, Oct 26, 2026*

Study: ashishps1 — Cache Eviction Policies; InterviewReady's Caching guide (their actual caching resources — no separate "Twitter cache paper" exists there, corrected)

- Worked Example: reproduce LRU/LFU eviction mechanics with a walkthrough

`Commit: hld-day-20-cache-eviction`

Day 21 — [WE3] Distributed Caching

Deadline: *Tuesday, Oct 27, 2026*

Study: ashishps1 — Distributed Caching; InterviewReady caching links

- Worked Example: reproduce Redis vs Memcached decision framework — when to pick which

`Commit: hld-day-21-distributed-caching`

Day 22 — [PT1 + PT2] Product Catalog: Cache Strategy + CDN Split

Deadline: *Wednesday, Oct 28, 2026*

Read: ashishps1 — Read-Through vs Write-Through Cache tradeoff page

- PT1: pick read-through vs write-through for the product catalog
- PT2: decide which catalog assets (images, static data) go through CDN vs cache

`Commit: hld-day-22-catalog-cache-strategy`

Day 23 — [PT3 + CH1] Product Catalog: Case Study + Build Challenge

Deadline: *Thursday, Oct 29, 2026*

Read: 1 more real caching case study (Uber's Integrated Cache, from InterviewReady links)

- PT3: apply the case study's insight — what does a "textbook" cache miss for this catalog?
- CH1 (practical build): build a Memcached CLI client as a coding challenge (verified — codingchallenges.fyi's "Build Your Own Memcached CLI Client" challenge; LRU eviction logic is exactly what you'll implement inside it)

`Commit: hld-day-23-catalog-case-study-build`

Day 24 — [CH2 + CH3 + Deliverable] Product Catalog: Invalidation + Stampede

Deadline: *Friday, Oct 30, 2026*

- CH2 (unguided): design cache invalidation strategy for catalog price updates
- CH3 (unguided): handle a cache-stampede/thundering-herd scenario during a flash sale
- **Deliverable:** full caching layer design doc for the Product Catalog (strategy + eviction policy + invalidation + stampede handling)

```
Commit: hld-day-24-catalog-caching-deliverable
```

WEEK 5 — Async, Messaging & Distributed Systems (Days 25-30)

Milestone Deadline: Saturday, Oct 31, 2026 → Friday, Nov 06, 2026

Target system for PT/CH/Deliverable: Order-Processing Async Pipeline

Day 25 — [WE1] Message Queues + Pub/Sub

Deadline: *Saturday, Oct 31, 2026*

Study: system-design-primer — Asynchronism section; ashishps1 — Pub/Sub, Message Queues

- Worked Example: reproduce when Message Queue fits vs when Pub/Sub fits

`Commit: hld-day-25-mq-pubsub`

Day 26 — [WE2] Change Data Capture + Event-Driven Basics

Deadline: *Monday, Nov 02, 2026*

Study: ashishps1 — Change Data Capture page

- Worked Example: reproduce how CDC keeps a search index in sync with a primary DB

`Commit: hld-day-26-cdc-event-driven`

Day 27 — [WE3] Heartbeats + Service Discovery

Deadline: *Tuesday, Nov 03, 2026*

Study: ashishps1 — HeartBeats, Service Discovery pages

- Worked Example: reproduce how a service registry detects and routes around a dead node

`Commit: hld-day-27-heartbeats-service-discovery`

Day 28 — [PT1 + PT2] Order Pipeline: Queue Structure + Leader Election

Deadline: *Wednesday, Nov 04, 2026*

Read: ashishps1 — Consensus Algorithms page; InterviewReady — Paxos, Raft links

- PT1: design the queue + retry structure for order events
- PT2: decide where (if anywhere) leader election/consensus is needed in the pipeline

`Commit: hld-day-28-order-pipeline-queue-consensus`

Day 29 — [PT3 + CH1] Order Pipeline: Locking + Consensus Challenge

Deadline: *Thursday, Nov 05, 2026*

Read: ashishps1 — Distributed Locking, Gossip Protocol pages

- PT3: add distributed locking for the inventory-decrement step
- CH1 (harder — background reading from CS-Papers-For-System-Design): derive why/where the pipeline actually needs Paxos/Raft-style consensus, before checking ashishps1's answer

`Commit: hld-day-29-order-pipeline-locking-consensus`

Day 30 — [CH2 + CH3 + Deliverable] Order Pipeline: Gossip + Circuit Breaker

Deadline: *Friday, Nov 06, 2026*

Read: ashishps1 — Circuit Breaker, Distributed Tracing, Disaster Recovery pages

- CH2 (unguided): design failure detection across pipeline worker nodes using gossip protocol
- CH3 (unguided): design a circuit breaker + fallback for a flaky payment-service dependency
- **Deliverable:** full async pipeline design (queue + retry + circuit breaker + tracing points) for the Order-Processing system

`Commit: hld-day-30-order-pipeline-deliverable`

WEEK 6 — Architecture Patterns + Tradeoffs (Days 31-36)

Milestone Deadline: Saturday, Nov 07, 2026 → Friday, Nov 13, 2026

Target system for PT/CH/Deliverable: any 1 sample system of your choice

Day 31 — [WE1] Client-Server + Microservices Architecture

Deadline: *Saturday, Nov 07, 2026*

Study: ashishps1 — Client-Server Architecture, Microservices Architecture pages

- Worked Example: reproduce when microservices actually pay off vs when they don't

```
Commit: hld-day-31-client-server-microservices
```

Day 32 — [WE2] Serverless + Event-Driven Architecture

Deadline: *Monday, Nov 09, 2026*

Study: ashishps1 — Serverless Architecture, Event-Driven Architecture pages

- Worked Example: reproduce a serverless vs always-on tradeoff with a concrete workload example

```
Commit: hld-day-32-serverless-event-driven
```

Day 33 — [WE3] P2P Architecture + Pattern Selection

Deadline: *Tuesday, Nov 10, 2026*

Study: ashishps1 — Peer-to-Peer Architecture page; cross-reference essential-js-design-patterns to connect architecture-level patterns back to your LLD design patterns

- Worked Example: for each architecture pattern learned this week, note 1 real product example

```
Commit: hld-day-33-p2p-pattern-selection
```

Day 34 — [PT1 + PT2] Tradeoffs Part 1

Deadline: *Wednesday, Nov 11, 2026*

Read: ashishps1 — Top 15 Tradeoffs page

- PT1: apply Vertical vs Horizontal Scaling to your chosen sample system
- PT2: apply Concurrency vs Parallelism to your chosen sample system

```
Commit: hld-day-34-tradeoffs-1
```

Day 35 — [PT3 + CH1] Tradeoffs Part 2

Deadline: *Thursday, Nov 12, 2026*

Read: ashishps1 — Long Polling vs WebSockets, Batch vs Stream Processing pages

- PT3: apply Long Polling vs WebSockets + Batch vs Stream to the system
- CH1 (unguided): redesign a stateful component of the system as stateless, before checking the Stateful vs Stateless page

```
Commit: hld-day-35-tradeoffs-2
```

Day 36 — [CH2 + CH3 + Deliverable] Tradeoffs Part 3

Deadline: *Friday, Nov 13, 2026*

Read: ashishps1 — Strong vs Eventual Consistency, Push vs Pull, REST vs RPC, Sync vs Async, Latency vs Throughput pages

- CH2 (unguided): pick + justify Strong vs Eventual Consistency and Push vs Pull for the system
- CH3 (unguided): pick + justify REST vs RPC, Sync vs Async, Latency vs Throughput
- **Deliverable:** 1-page tradeoff decision doc covering at least 5 tradeoffs for the chosen sample system

Commit: hld-day-36-tradeoffs-deliverable

WEEK 7 — Easy Interview Problems, Hands-On (Days 37-42)

Milestone Deadline: Saturday, Nov 14, 2026 → Friday, Nov 20, 2026

Har din: full interview format — Requirements → Back-of-envelope estimates → API design → High-level diagram → Deep dive → Tradeoffs. Ek din = ek hi system.

Day 37 — [WE1] Design URL Shortener (TinyURL)

Deadline: *Saturday, Nov 14, 2026*

Reference: system-design-primer's Pastebin solution (same pattern)

- Worked Example: study the Pastebin solution, then reproduce it as URL Shortener in your own words/diagram

```
Commit: hld-day-37-url-shortener
```

Day 38 — [WE2] Design Rate Limiter (full interview format)

Deadline: *Monday, Nov 16, 2026*

Reference: ashishps1's Medium list (verified — Rate Limiter sits under Medium there, tackled early since you already built one in Week 2); use 30-seconds-of-code for quick algorithm snippet reference

- Worked Example: full guided design, closely following the reference structure

```
Commit: hld-day-38-rate-limiter-full-design
```

Day 39 — [WE3] Design Parking Garage

Deadline: *Tuesday, Nov 17, 2026*

- Revisit your LLD Parking Lot (Day 23 of the LLD roadmap) — add scale/infra layer (multiple locations, real-time availability sync)
- Worked Example: guided reproduction using LLD base + HLD scale layer

```
Commit: hld-day-39-parking-garage-scale
```

Day 40 — [PT1 + PT2] Design Distributed Key-Value Store / Cache

Deadline: *Wednesday, Nov 18, 2026*

Reference: system-design-primer's key-value store links (lighter guidance than WE days)

- PT1: data partitioning decision for the KV store
- PT2: replication/consistency decision for the KV store

```
Commit: hld-day-40-distributed-kv-store
```

Day 41 — [PT3 + CH1] Design Authentication System

Deadline: *Thursday, Nov 19, 2026*

Reference: InterviewReady — Designing an Authorization Model for an Enterprise

- PT3: core auth system design (light guidance)
- CH1 (unguided): token revocation at scale — solve before checking any reference

```
Commit: hld-day-41-authentication-system
```

Day 42 — [CH2 + CH3 + Deliverable] Design Web Crawler (Review Day)

Deadline: *Friday, Nov 20, 2026*

Reference: system-design-primer's Web Crawler solution

- CH2 (unguided): dedup at scale — solve before checking the reference
- CH3 (unguided): distributed politeness policy across crawler nodes — solve before checking
- **Deliverable:** polish the Web Crawler design into a full interview-format write-up

- Revision: self-test all 6 Easy designs this week — could you re-draw each from memory?

```
Commit: hld-day-42-web-crawler-review
```

WEEK 8 — Medium + Hard, Capstone (Days 43-48)

Milestone Deadline: Saturday, Nov 21, 2026 → Friday, Nov 27, 2026

Day 43 — [WE1] Design WhatsApp / Chat App

Deadline: *Saturday, Nov 21, 2026*

Reference: ashishps1's "Design WhatsApp" link (verified — this one isn't in system-design-primer's 8 solved problems, only in ashishps1's Medium tier)

- Worked Example: guided study + reproduction

```
Commit: hld-day-43-whatsapp
```

Day 44 — [WE2] Design Instagram/Twitter Feed

Deadline: *Monday, Nov 23, 2026*

Reference: system-design-primer's Twitter solution (fanout, timeline generation)

- Worked Example: guided study + reproduction

```
Commit: hld-day-44-social-feed
```

Day 45 — [WE3] Design Netflix / YouTube

Deadline: *Tuesday, Nov 24, 2026*

Reference: ashishps1 Medium list

- Worked Example: guided study + reproduction

```
Commit: hld-day-45-video-streaming
```

Day 46 — [PT1 + PT2] Design Notification & Scheduling System

Deadline: *Wednesday, Nov 25, 2026*

Reference: ashishps1 Medium list *(single combined system: notification service uses the scheduler as its delivery mechanism)*

- PT1: requirements + API design for the notification & scheduling system
- PT2: architecture (queue + scheduler + delivery workers)

```
Commit: hld-day-46-notification-job-scheduler
```

Day 47 — [PT3 + CH1] Design Uber — Part 1 (Capstone)

Deadline: *Thursday, Nov 26, 2026*

Reference: ashishps1 Hard list; optionally cross-check problem framing against grokking-system-design

- PT3: requirements, back-of-envelope estimates, API design (light guidance)
- CH1 (unguided): draft the high-level architecture before checking any reference

```
Commit: hld-day-47-uber-capstone-draft
```

Day 48 — [CH2 + CH3 + Deliverable] Design Uber — Part 2, Capstone Finalize

Deadline: *Friday, Nov 27, 2026*

- CH2 (unguided): deep-dive challenge — matching algorithm at scale
- CH3 (unguided): deep-dive challenge — real-time location tracking at scale
- **Deliverable:** clean up into a polished write-up (README-style, same format as your LLD Task Manager capstone)
- Practice presenting the capstone in interview format — use tech-interview-handbook for structure/behavioral polish
- Add it to the same GitHub portfolio as the LLD capstone — "LLD + HLD" combined shows full range

```
Commit: hld-day-48-capstone-final
```

Reference Section

Weekly Coverage Map

Week	Topic	Target System	Primary Source
1	Foundations + Core Concepts	Distributed Session Store	system-design-primer, what-happens-when
2	Networking & API Layer	Ride-Hailing App API	system-design-primer, ashishps1, CodingChallengesFYI
3	Database Layer	Ride-Tracking App DB	system-design-primer
4	Caching Layer	Product Catalog Cache	ashishps1, InterviewReady, awesome-scalability, CodingChallengesFYI
5	Async + Distributed Systems	Order-Processing Pipeline	ashishps1, InterviewReady, CS-Papers-For-System-Design
6	Architecture Patterns + Tradeoffs	Chosen sample system	ashishps1, essential-js-design-patterns
7	Easy Interview Problems	6 separate systems (1/day)	system-design-primer, ashishps1, 30-seconds-of-code
8	Medium/Hard + Capstone	4 systems + Uber capstone	system-design-primer, ashishps1, grokking-system-design, tech-interview-handbook

Full Repo List (all 15, from your image — every repo now has a role)

1. design-gurus/grokking-system-design — optional extra problem picks, Weeks 7-8 (verified real repo; corrected from image's "design-guru")
2. yangshun/tech-interview-handbook — capstone presentation polish, Day 48
3. kdn251/interviews — parallel-track DSA prep (outside the 2hr HLD block)
4. alex/what-happens-when — Day 1 worked example
5. Chalarangelo/30-seconds-of-code — quick snippets for API contracts, Weeks 2 & 7
6. codersguild/System-Design — backup chapter-wise reading
7. arpit20adlakha/Computer-Science-Papers-For-System-Design — Week 5 Challenge background reading
8. InterviewReady/system-design-resources — Week 4 (caching), Week 5 (consensus), Week 7 (auth)
9. arpit20adlakha/Data-Structure-Algorithms-LLD-HLD — backup index, as-needed
10. kilimchoi/engineering-blogs — alt case-study source, Weeks 1-6
11. donnemartin/system-design-primer — primary backbone, Weeks 1-8
12. addyosmani/essential-js-design-patterns — Week 6, architecture ↔ LLD pattern connection
13. binhnguyennus/awesome-scalability — case studies, Weeks 1-6
14. ashishps1/awesome-system-design-resources — primary index, Weeks 1-8
15. CodingChallengesFYI/SharedSolutions — Challenge builds, Week 2 (rate limiter), Week 4 (LRU cache)

Interview Format Checklist (use every day in Weeks 7-8)

- Requirements — functional + non-functional, scope clarified
- Back-of-envelope estimates — traffic, storage, bandwidth
- API design — key endpoints/contracts
- High-level diagram — major components
- Deep dive — 1-2 components in detail (the "interesting" part)

- Tradeoffs — what you'd reconsider at 10x scale